

CẤU TRÚC DỮ LIỆU & GIẢI THUẬT

BẢNG BẮM – HÀM BẮM



TS. TRẦN NGỌC VIỆT
TH.S LÊ CÔNG HIẾU



HỌC KỲ 3 – NĂM HỌC 2025-2026



KHÓA K31 & K30



NỘI DUNG

01. GIỚI THIỆU

02. BẢNG BĂM

03. HÀM BĂM

04. XUNG ĐỘT

05. BÀI TẬP



GIỚI THIỆU

- Tổ chức lưu trữ thông tin 100 nhân viên của một công ty ABC, mỗi nhân viên có mã số riêng EMP_ID trong phạm vi [0, 99]
- Ta có thể dùng mảng để lưu trữ với EMP_ID là chỉ mục tương ứng trong mảng.

Dữ liệu →

EMP_ID →

An	Bình	Minh	...	Ngọc
0	1	2	...	99

GIỚI THIỆU

- Tổ chức lưu trữ thông tin 100 nhân viên của một công ty ABC, mỗi nhân viên có mã số riêng EMP_ID trong phạm vi [00000, 99999]
- Ta cần một mảng có 100,000 phần tử để lưu trữ với EMP_ID là chỉ mục tương ứng trong mảng.

Dữ liệu →	An	Bình	Minh	...	Ngọc	...	
EMP_ID →	00000	00001	00002	...	99	...	99999

→ Tốn nhiều không gian lưu trữ



GIỚI THIỆU

- Cần giải pháp khác để lưu trữ 100 nhân viên với EMP_ID có phạm vi [00000, 99999]
- Cần có hàm chuyển EMP_ID có 5 số về EMP_ID có 2 số (hàm băm)
- Cần có mảng ánh xạ mỗi khóa EMP_ID 5 số tới vị trí trong mảng EMP_ID 2 số. (Bảng băm)

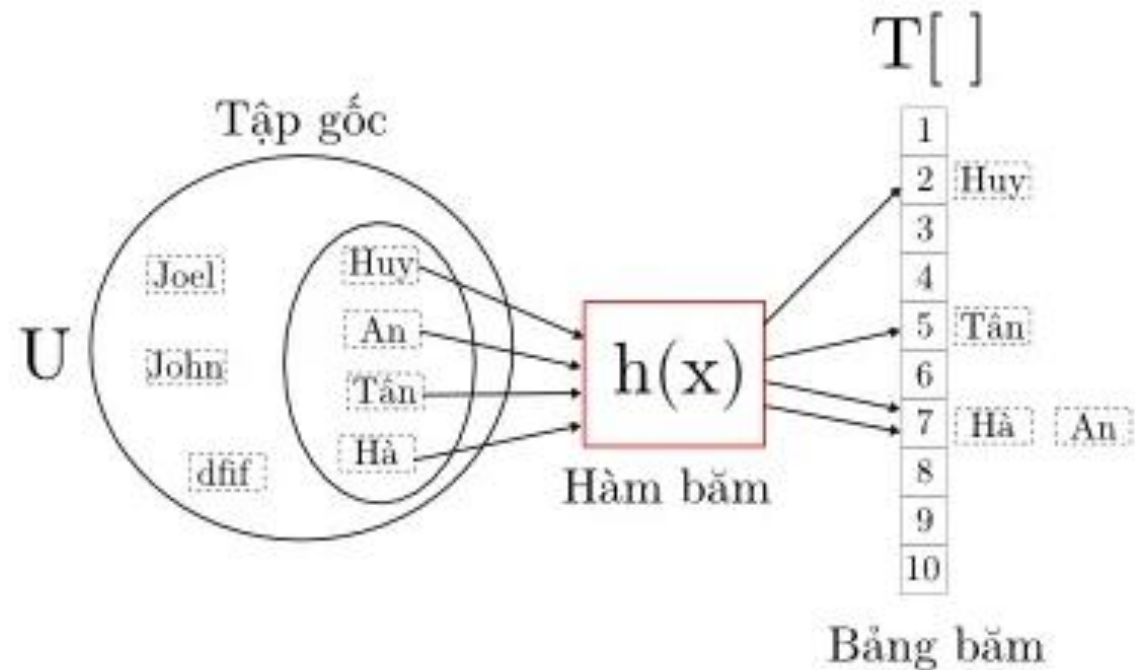


BẢNG BẮM

- Bảng băm là một cấu trúc dữ liệu mà các khóa được ánh xạ đến các vị trí trong mảng thông qua một hàm băm.
- Trong bảng băm, một phần tử có khóa k được lưu tại chỉ mục có hàm băm $h(k)$ chứ không phải vị trí k .
- Quá trình ánh xạ các khóa vào vị trí phù hợp trong bảng băm gọi là hashing.
- Khi hai khóa có cùng vị trí trong bảng băm gọi là xung đột (collision)

BẢNG BĂM

- Bảng băm là một cấu trúc dữ liệu mà các khóa được ánh xạ đến các vị trí trong mảng thông qua một hàm băm.





HÀM BẮM

- Hàm băm là một công thức toán học khi áp dụng ánh xạ cho khóa k sẽ tạo ra một số nguyên dùng làm chỉ mục trong bảng băm.
- Một hàm băm thỏa mãn các điều kiện sau:
 1. Tính toán nhanh
 2. Các khóa được phân bố đều trong bảng
 3. Ít xảy ra xung đột
 4. Xử lý các loại khóa có kiểu dữ liệu khác nhau



CÁC PHƯƠNG PHÁP TẠO HÀM BĂM

1. Phương pháp chia

$$h(x) = x \% M$$

VD: tính giá trị băm cho các khóa 1234 ; 5462, 2362

$M = 10$ (chọn số chẵn – kích thước bảng)

$$h(1234) = 1234 \% 10 = 4$$

$$h(5462) = 5462 \% 10 = \mathbf{2}$$

$$h(2362) = 2362 \% 10 = \mathbf{2}$$

Xung đột

CÁC PHƯƠNG PHÁP TẠO HÀM BĂM

1. Phương pháp chia

$$h(x) = x \% M$$

VD: tính giá trị băm cho các khóa 1234 ; 5462, 2362

$M = 5$ (Chọn số lẻ – kích thước bảng)

$$h(1234) = 1234 \% 5 = 4$$

$$h(5462) = 5462 \% 5 = \mathbf{2}$$

$$h(2362) = 2362 \% 5 = \mathbf{2}$$

Xung đột



CÁC PHƯƠNG PHÁP TẠO HÀM BĂM

1. Phương pháp chia

$$h(x) = x \% M$$

VD: tính giá trị băm cho các khóa 1234 ; 5462, 2362

$M = 97$ (Chọn số nguyên tố – gần kích thước bảng)

$$h(1234) = 1234 \% 97 = 70$$

$$h(5462) = 5462 \% 97 = 30$$

$$h(2362) = 2362 \% 97 = 34$$

Không xung đột



CÁC PHƯƠNG PHÁP TẠO HÀM BẮM

1. Phương pháp chia

- Việc chọn M là số nguyên tố giúp tạo các khóa phân bố đồng nhất.
- M là số nguyên tố gần với 2^k vì nếu chọn bằng 2^k :

$$h(x) = x \% 2^k$$

Khi đó $h(x)$ sẽ chọn giá trị là k bit cuối cùng của x.

Ví dụ: $x = 15$ (1111), $M = 8$ (2^3)

→ $h(15) = 15 \% 8 = 7$ (111)



CÁC PHƯƠNG PHÁP TẠO HÀM BẮM

2. Phương pháp nhân

- Thực hiện 4 bước

B1 : Chọn một hằng số A sao cho $0 < A < 1$

B2 : Nhân khóa $x * A$

B3 : Trích phần phân số của $x * A$

B4 : Nhân kết quả B3 với M là kích thước bảng băm

CÁC PHƯƠNG PHÁP TẠO HÀM BẮM

2. Phương pháp nhân

$$h(x) = M * (x * A \bmod 1)$$

Trong đó,

$(x * A \bmod 1)$ trích phần thập phân của $x * A$

M là kích thước bảng băm

CÁC PHƯƠNG PHÁP TẠO HÀM BẮM

2. Phương pháp nhân

$$h(x) = M * (x * A \bmod 1)$$

Chọn A phù hợp để hàm băm hiệu quả

A : thường chọn theo đề xuất của Knuth là tốt nhất

$$A = \frac{\sqrt{5}-1}{2} = 0.618033$$

CÁC PHƯƠNG PHÁP TẠO HÀM BĂM

2. Phương pháp nhân

$$h(x) = M * (x * A \bmod 1)$$

Ví dụ: Cho $A = 0.618033$; $M = 1000$. Tính $x = 12345$

$$h(12345) = 1000 * (12345 * 0.618033 \bmod 1)$$

$$= 1000 * (7629.617385 \bmod 1)$$

$$= 1000 * (0.617385)$$

$$= 617.385 = 617$$

CÁC PHƯƠNG PHÁP TẠO HÀM BẮM

(1,20)
(2,70)
(42,80)
(4,25)
(12,44)
(14,32)
(17,11)
(13,78)
(37,98)

Stt	Key	Hash	Chỉ mục mảng
1	1	$1 \% 20 = 1$	1
2	2	$2 \% 20 = 2$	2
3	42	$42 \% 20 = 2$	2
4	4	$4 \% 20 = 4$	4
5	12	$12 \% 20 = 12$	12
6	14	$14 \% 20 = 14$	14
7	17	$17 \% 20 = 17$	17
8	13	$13 \% 20 = 13$	13
9	37	$37 \% 20 = 17$	17

CÁC PHƯƠNG PHÁP TẠO HÀM BẮM

Kỹ thuật Dò tuyến tính - Linear Probing

-Điều này thấy rằng có thể xảy ra trường hợp mà kỹ thuật Hashing được sử dụng để tạo chỉ mục đã tồn tại trong mảng.

-Tình huống, cần tìm kiếm vị trí trống kế tiếp trong mảng bằng việc nhìn vào trong ô tiếp theo cho tới khi chúng ta tìm thấy một ô trống. Kỹ thuật này được gọi là Dò tuyến tính.

Stt	Key	Hash	Chỉ mục mảng	Sau kỹ thuật Linear Probing, chỉ mục mảng
1	1	$1 \% 20 = 1$	1	1
2	2	$2 \% 20 = 2$	2	2
3	42	$42 \% 20 = 2$	2	3
4	4	$4 \% 20 = 4$	4	4
5	12	$12 \% 20 = 12$	12	12
6	14	$14 \% 20 = 14$	14	14
7	17	$17 \% 20 = 17$	17	17
8	13	$13 \% 20 = 13$	13	13
9	37	$37 \% 20 = 17$	17	18

BÀI KIỂM TRA SỐ 7

BàiTap 01 - Phương pháp chia

$$h(x) = x \% M$$

Tính giá trị băm cho các khóa 15, 11, 27, 8, 32

$M = 7$ (Chọn số lẻ – kích thước bảng)

BàiTap 02 - Phương pháp chia

$$h(x) = x \% M$$

VD: tính giá trị băm cho các khóa 1234 ; 5462, 2362

$M = 5$ (Chọn số lẻ – kích thước bảng)

BàiTap 03 - Phương pháp chia

$$h(x) = x \% M$$

Tính giá trị băm cho các khóa 10 ; 25, 20; 9; 21; 21

$M = 10$ (kích thước bảng)





#Thực hành 01:

```
>>> def hash_key( key, m):          #??
    return (key % m)                #??

>>> m = 7

>>> print(f'The hash value is {hash_key(15,m)}') #??
>>> print(f'The hash value is {hash_key(2,m)}')
>>> print(f'The hash value is {hash_key(3,m)}')
>>> print(f'The hash value is {hash_key(9,m)}')
>>> print(f'The hash value is {hash_key(11,m)}')
>>> print(f'The hash value is {hash_key(7,m)}')
```

#Nhận xét: kĩ thuật Dò tuyến tính nếu có Xung đột ?

#Thực hành 02:

```
>>> def display_hash(hashTable):                                #??
    for i in range(len(hashTable)):
        print(i, end = " ")

        for j in hashTable[i]:
            print("-->", end = " ")
            print(j, end = " ")

        print()

>>> HashTable = [[] for _ in range(10)]                        #??
>>> def Hashing(keyvalue):                                     #??
    return (keyvalue % len(HashTable))

>>> def insert(Hashtable, keyvalue, value):                   #??

    hash_key = Hashing(keyvalue)
    Hashtable[hash_key].append(value)

>>> insert(HashTable, 10, 'MachineLearning')                 #??
>>> insert(HashTable, 45, 'DataScience')
>>> insert(HashTable, 20, 'DataAnalytics')
>>> insert(HashTable, 9, 'BigData')
>>> insert(HashTable, 21, 'DataStructure ')
>>> insert(HashTable, 41, 'IoT')
>>> insert(HashTable, 35, 'Probability')
>>> display_hash (HashTable)                                  #??
#nhan xet
```

#Thực hành 03:

```
class Node:
    def __init__(self, key, value):
        self.key = key
        self.value = value
        self.next = None

class HashTable:
    def __init__(self, capacity):
        self.capacity = capacity
        self.size = 0
        self.table = [None] * capacity

    def _hash(self, key):
        return hash(key) % self.capacity

    def insert(self, key, value):
        index = self._hash(key)

        if self.table[index] is None:
            self.table[index] = Node(key, value)
            self.size += 1
        else:
            current = self.table[index]
            while current:
                if current.key == key:
                    current.value = value
                    return
                current = current.next
            new_node = Node(key, value)
            new_node.next = self.table[index]
            self.table[index] = new_node
            self.size += 1
```

#Thực hành 03:

```
def search(self, key):
    index = self._hash(key)

    current = self.table[index]
    while current:
        if current.key == key:
            return current.value
        current = current.next
    raise KeyError(key)

def remove(self, key):
    index = self._hash(key)
    previous = None
    current = self.table[index]
    while current:
        if current.key == key:
            if previous:
                previous.next = current.next
            else:
                self.table[index] = current.next
            self.size -= 1
            return
        previous = current
        current = current.next
    raise KeyError(key)

def __len__(self):
    return self.size

def __contains__(self, key):
    try:
        self.search(key)
        return True
    except KeyError:
        return False
```

```
if __name__ == '__main__':
    #ham main

    ht = HashTable(7)
    #kich thuoc bang

    ht.insert("data science", 15)
    ht.insert("IoT", 11)
    ht.insert("machine learning", 27)
    ht.insert("deep learning", 8)
    ht.insert("computer science", 32)

    print("data science" in ht) # True
    print("IT" in ht) # False

    print(ht.search("IoT")) # 11

    ht.insert("big data", 18)
    print(ht.search("machine learning")) # 27
    ht.remove("IoT")

    #kiem tra kich thuoc
    bang
    print(len(ht)) # 5
```

#Thực hành 04:

```
>>> import math
>>> import sys
>>> class Graph():
    def __init__(cung, dinh):
        cung.x = dinh
        cung.graph = [[0 for column in range(dinh)]
                       for row in range(dinh)]
    def inketqua(cung, L, a):
        print ("đỉnh nguồn xuất phát từ: ")
        for nut in range(cung.x):
            print (a," đến đỉnh ",nut, "độ dài đường đi là: ", L[nut])
    def duongdinhonhat(cung, L, P):
        min = sys.maxsize
        for x in range(cung.x):
            if L[x] < min and P[x] == False:
                min = L[x]
                min_index = x
        return min_index
    def timduongdi(cung, a):
        L = [sys.maxsize] * cung.x
        L[a] = 0
        P = [False] * cung.x
        for cout in range(cung.x):
            u = cung.duongdinhonhat(L, P)
            P[u] = True
            for x in range(cung.x):
                if cung.graph[u][x] > 0 and P[x] == False and L[x] > L[u]
+ cung.graph[u][x]:
                    L[x] = L[u] + cung.graph[u][x]
            cung.inketqua(L, a)

>>> g = Graph(6)      #đồ thị vô hướng
>>> g.graph = [[0, 3, 0, 1, 0, 0],
               [3, 0, 5, 2, 0, 0],
               [0, 5, 0, 0, 4, 2],
               [1, 2, 0, 0, 6, 0],
               [0, 0, 4, 6, 0, 7],
               [0, 0, 2, 0, 7, 0]
               ];
>>> g.timduongdi(0);
```

#Nhận xét: input, output & vẽ đồ thị

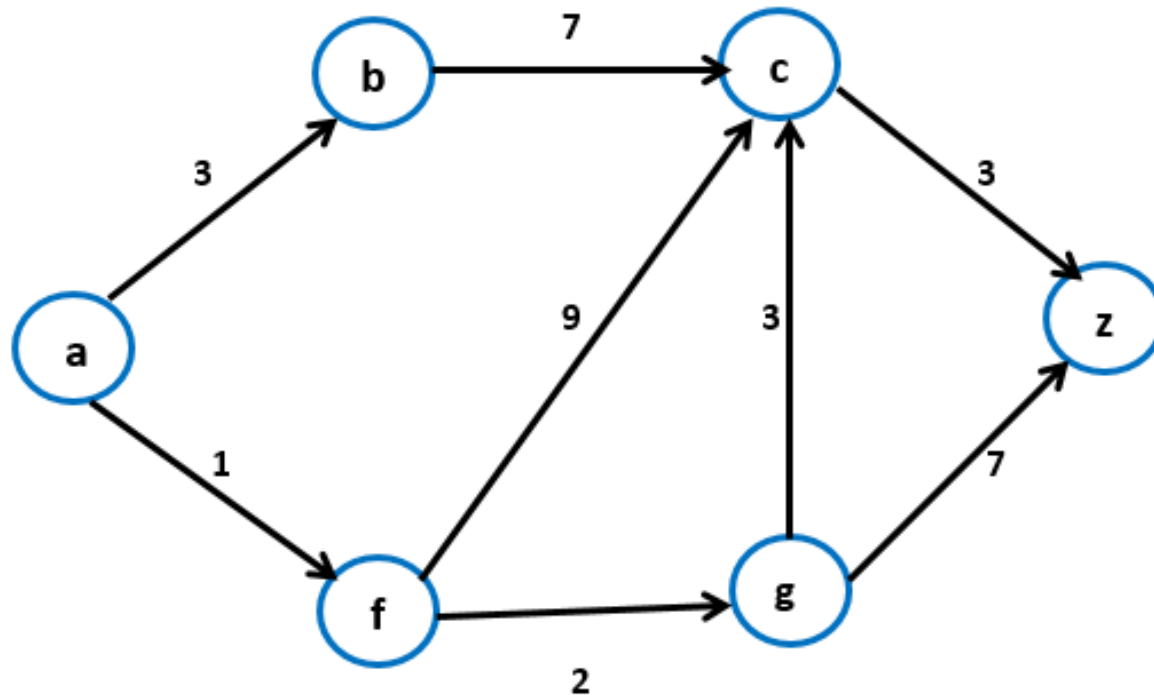
#Thực hành 05 (tương tự TH 04) :

```
>>> g = Graph(6)           #đồ thị có hướng
>>> g.graph = [[0, 3, 0, 4, 0, 0],
               [0, 0, 6, 2, 0, 0],
               [0, 0, 0, 0, 4, 3],
               [0, 0, 1, 0, 4, 0],
               [0, 0, 0, 0, 0, 5],
               [0, 0, 0, 0, 0, 0]
               ];
>>> g.timduongdi(0);
```

#Nhận xét: input, output & vẽ đồ thị

#Thực hành 06 (tương tự TH 04) :

- Viết ma trận kề từ đồ thị có hướng
- Tìm đường đi ngắn nhất từ đỉnh a đến đỉnh z trong đồ thị có hướng.

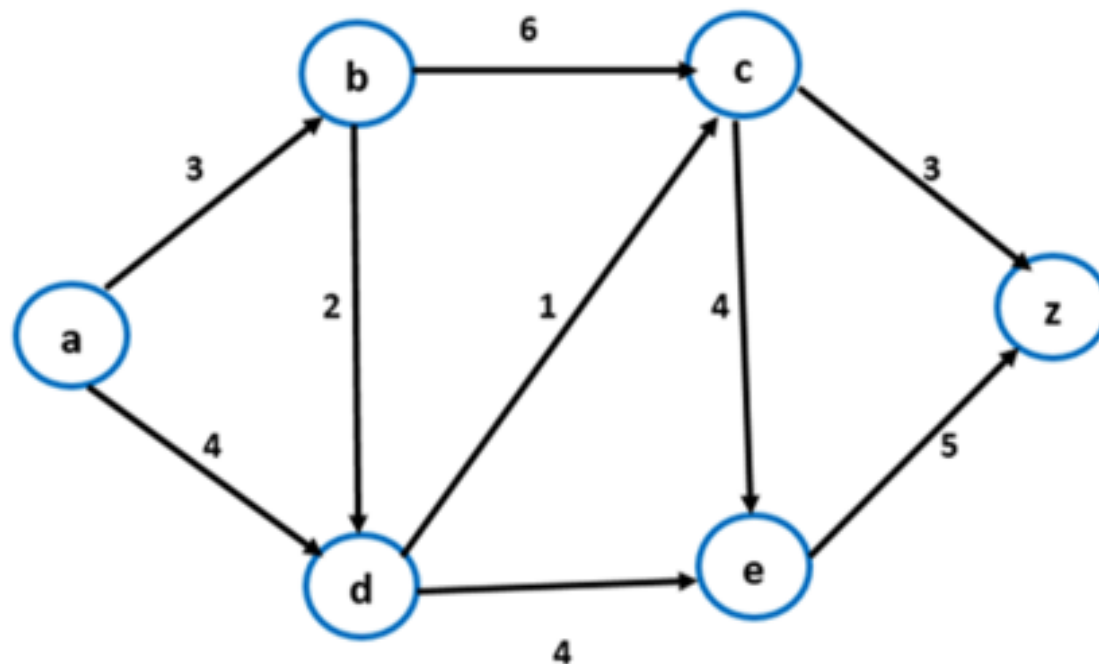


(Hình 1)

#Nhận xét: input, output & biểu diễn đường đi ngắn nhất

#Thực hành 07 (tương tự TH 04) :

- Viết ma trận kề từ đồ thị có hướng
- Tìm đường đi ngắn nhất từ đỉnh a đến đỉnh z trong đồ thị có hướng.



(Hình 1)

#Nhận xét: input, output & biểu diễn đường đi ngắn nhất



TRƯỜNG ĐẠI HỌC
VĂN LANG

Đạo đức - Ý chí - Sáng tạo

KHOA CÔNG NGHỆ THÔNG TIN

